COMILLAS UNIVERSIDAD PONTIFICIA ICAI ICADE CIHS

ASSIGNMENT 8.2

Chatbot de riesgo con LLM, FastAPI y un modelo

Construir el backend que conecta un LLM con el modelo joblib mediante una tool

DURACION	MODALIDAD	PREREQUISITOS
120 min	Individual, practica final	Semanas 1 a 5 (Pydantic, joblib, tests, Streamlit)

1. Encargo

Construye el backend de un chatbot de solicitudes de prestamo. El usuario escribe en lenguaje natural ("calcula el riesgo para 30 anos, ingresos 2500...") y por detras: el frontend Streamlit envia la conversacion a tu API, tu API FastAPI (`POST /chat`) la pasa a un LLM, el LLM decide llamar a una tool con los datos del prestamo, tu codigo valida esos datos con Pydantic, llama al modelo joblib y devuelve el resultado al LLM, y el LLM redacta la respuesta final.

Solo haces el backend. El frontend, el cliente del LLM (LiteLLM) y los tests ya vienen hechos.

2. Punto de partida

Recibes el proyecto con la estructura estandar de una app FastAPI (`app/` con `routers`, `schemas`, `services`; `frontend/`; `models/`; `scripts/`; `tests/`; `pyproject.toml`). Los `TODO` estan en cinco ficheros. Instala con `uv sync` (sin fijar version de Python).

3. El modelo

Recibe 6 variables, en este orden (esta en `VARIABLES`):

Variable	Tipo	Regla
edad	entero	de 18 a 75
ingresos_mensuales	decimal	mayor que 0
importe_prestamo	decimal	mayor que 0
plazo_meses	entero	de 6 a 84
antiguedad_laboral_anios	entero	de 0 a 50 y no mayor que edad - 16
impagos_previos	entero	de 0 a 20

`modelo.predict_proba([[...seis valores...]])[0][1]` es la probabilidad de impago. Decision: menor de 0,2 = `aprobar`; menor de 0,5 = `revisar`; resto = `rechazar`.

4. Tareas

01 app/schemas/prestamo.py - `PrestamoEntrada` con las 6 variables y sus reglas; rechaza campos inexistentes (el LLM a veces se inventa alguno) y valida la antiguedad con un `field_validator`. `PrediccionSalida`: `probabilidad_impago` (entre 0 y 1) y `decision`.

02 app/services/modelo.py - Carga el joblib una sola vez; `decidir()` y `predecir()`.

03 app/services/orquestador.py - `definir_tool()`: sus `parameters` salen de `PrestamoEntrada`, sin escribir los campos a mano. `ejecutar_tool(nombre, argumentos_json)`: valida con Pydantic y llama al modelo; devuelve siempre un dict con `"ok"` y, si algo falla (tool desconocida, JSON roto, dato invalido, campo inventado), `{"ok": False, "error": "..."}` en vez de lanzar excepcion. `responder(mensajes, llm)`: bucle LLM, tool, LLM, con mensaje `system` y nunca mas de `MAX_VUELTAS` vueltas.

04 app/routers/predict.py y chat.py - `POST /predict` y `POST /chat`; `/chat` recibe el LLM con `Depends(get_llm)` para que los tests lo sustituyan por uno falso.

05 Prueba - `uv run pytest` en verde y prueba el chatbot completo con el LLM simulado.

5. Entrega

Evidencia	Que debe permitir comprobar
Repositorio GitHub	Proyecto completo con los TODO resueltos.
uv run pytest	Todos los tests en verde, sin LLM y sin red.
Enlace a la rama o PR final	Entrega en tu fork (no en el repositorio oficial).

6. Criterios de aceptacion

[ ] `PrestamoEntrada` rechaza campos inventados y antiguedad incoherente con la edad.
[ ] La tool sale del esquema Pydantic; hay un unico contrato.
[ ] `ejecutar_tool` nunca lanza excepcion: devuelve ok True o False.
[ ] El bucle del orquestador respeta `MAX_VUELTAS`.
[ ] `/predict` y `/chat` funcionan y los tests pasan.

7. Comprobacion

```
# Tests (sin LLM, sin red)
uv sync
uv run pytest

# Chatbot completo (dos terminales Git Bash)
uv run uvicorn app.main:app --port 8000
uv run streamlit run frontend/app.py

# LLM real (opcional): LLM_MODEL=azure/<deployment> u ollama/<modelo>
# Los datos del LLM real los da el profesor en clase.
```

8. Extra opcional

Un `Dockerfile` para la API y un `docker-compose.yml` que levante `api` y `streamlit` (el frontend lee la variable `API_URL`).

9. Notas y limites

No modifiques `llm.py`, el frontend ni los tests. Dependencias sin versiones. Si el joblib da error de version de scikit-learn, ejecuta `scripts/entrenar_modelo.py`.

Referencia en el repositorio
semana8/modules/08-chatbot-llm-api/exercises/01-chatbot-riesgo-impago/problem/ENUNCIADO_ALUMNOS.md